МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Казанский национальный исследовательский технический
университет им. А.Н. Туполева – КАИ»
Институт компьютерных технологий и защиты информации
Кафедра Систем Информационной Безопасности

Лабораторная работа № 3

Дисциплина: «Аппаратные средства телекоммуникационных систем»

Тема: «Параллельное вычисление геометрического центра набора 3D-точек с использованием OpenMP»

Работу выполнил
Студент гр 4404
Завалин М.Г.

Казань, 2026

Цель работы: Реализовать две стратегии распараллеливания для одной и той же задачи: распараллеливание по данным (цикл) и функциональную декомпозицию (секции), сравнить их корректность и особенности использования.

1. Задание

Напишите программу, которая читает из файла координаты точек в 3D пространстве (x,y,z) и вычисляет геометрический центр, который есть среднее по x, y и z. Напишите две версии программы: одну с расспараелеливанием цикла, другую с функциональной декомпозицией.

2. Ход работы

В ходе выполнения лабораторной работы были реализованы 2 варианта выполнения задачи: 1 вариант - расспараелеливание цикла, 2 вариант - функциональная декомпозиция.

Вариант 1: Задача разбивается по данным: каждый поток обрабатывает свой диапазон индексов массива. Для безопасного суммирования используется редукция.

Работа при 1 варианте (расспараелеливание цикла) представлена на рисунке 1.

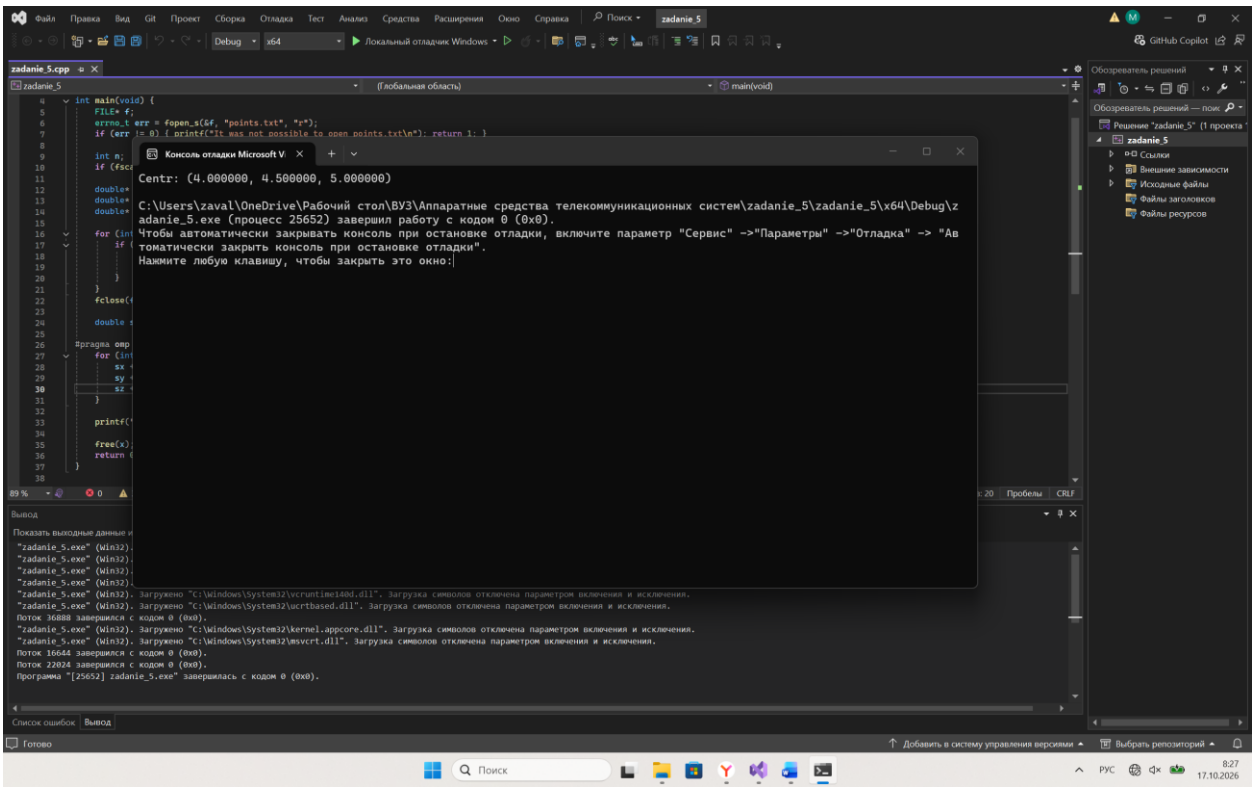


Рисунок 1 — запуск программы с расспараелеливанием цикла

Вариант 2: Работа разделяется по функциям: один поток суммирует x, другой — y, третий — z. Для этого используется конструкция sections.

Работа при 2 варианте (функциональная декомпозиция) представлена на рисунке 2.

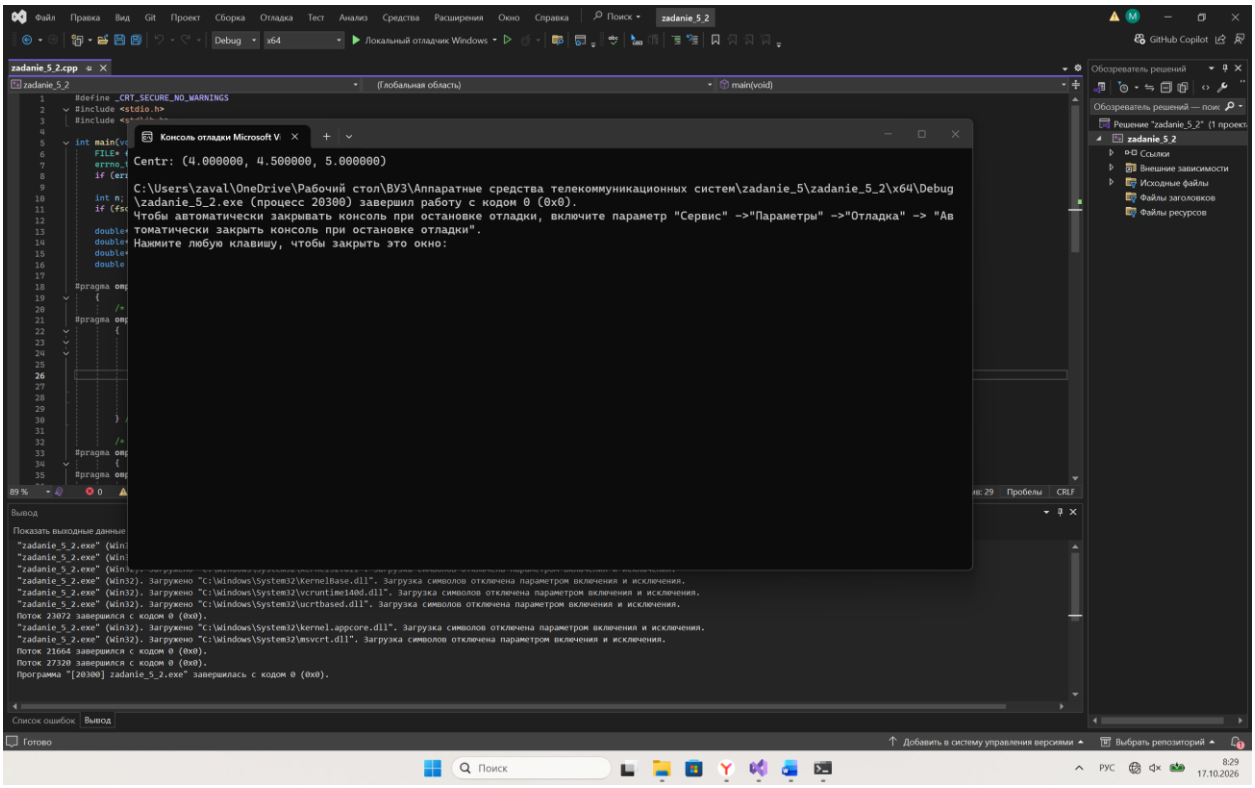


Рисунок 2 — запуск программы с функциональной декомпозиции

Вывод: В первом варианте работа делится по данным: потоки параллельно обрабатывают разные участки массива, а суммы безопасно объединяются через редукцию — это проще и эффективнее.

Во втором варианте работа делится по функциям: каждый поток считает сумму по своей координате (x/y/z), а объединение делается вручную через critical — так нагляднее демонстрируется синхронизация, но есть лишние накладные расходы и выше риск ошибок.